

OBSERVACIONES DE LA PRÁCTICA

202615358 Larry Thomas Sanchez
202522281 Jorge Claros

Máquina 1	
Procesador	AMD Ryzen 7 7800X3D
Memoria RAM (GB)	32 GB RAM
Sistema Operativo	Windows 11

Tabla 1. Especificaciones de la máquina para ejecutar las pruebas de rendimiento.

Resultados

Carga de Catálogo LINEAR PROBING

Factor de Carga (PROBING)	Consumo de Datos [kB]	Tiempo de Ejecución Real @LP [ms]
0.1	352313,42	3231,29
0.5	359033,09	3676,05
0.7	342313,12	3723,23
0.9	349423,98	3999,91

Tabla 2. Comparación de consumo de datos y tiempo de ejecución para carga de catálogo con el índice por categorías utilizando LINEAR PROBING

Carga de Catálogo SEPARATE CHAINING

Factor de Carga (CHAINING)	Consumo de Datos [kB]	Tiempo de Ejecución Real @SC [ms]
2.00	602779,1	4103,2
4.00	580100,90	4302,98
6.00	579213,32	4501,54
8.00	579065,76	4559,35

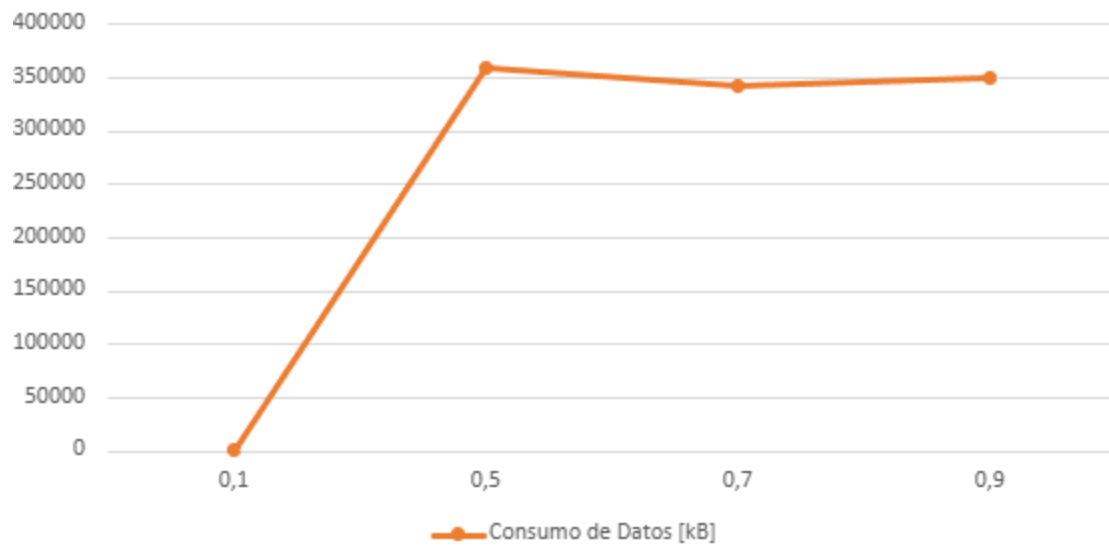
Tabla 3. Comparación de consumo de datos y tiempo de ejecución para carga de catálogo con el índice por categorías utilizando SEPARATE CHAINING

Gráficas

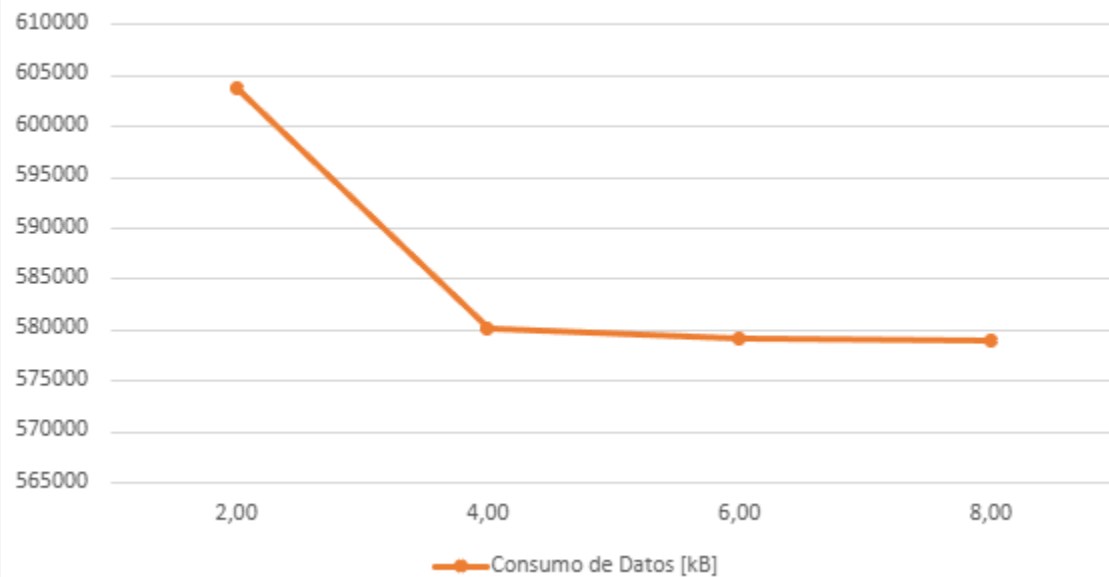
La gráfica generada por los resultados de las pruebas de rendimiento.

- Comparación de memoria y tiempo de ejecución para LINEAR PROBING y SEPARATE CHAINING
- Comparación de factor de carga y memoria para LINEAR PROBING y SEPARATE CHAINING
-

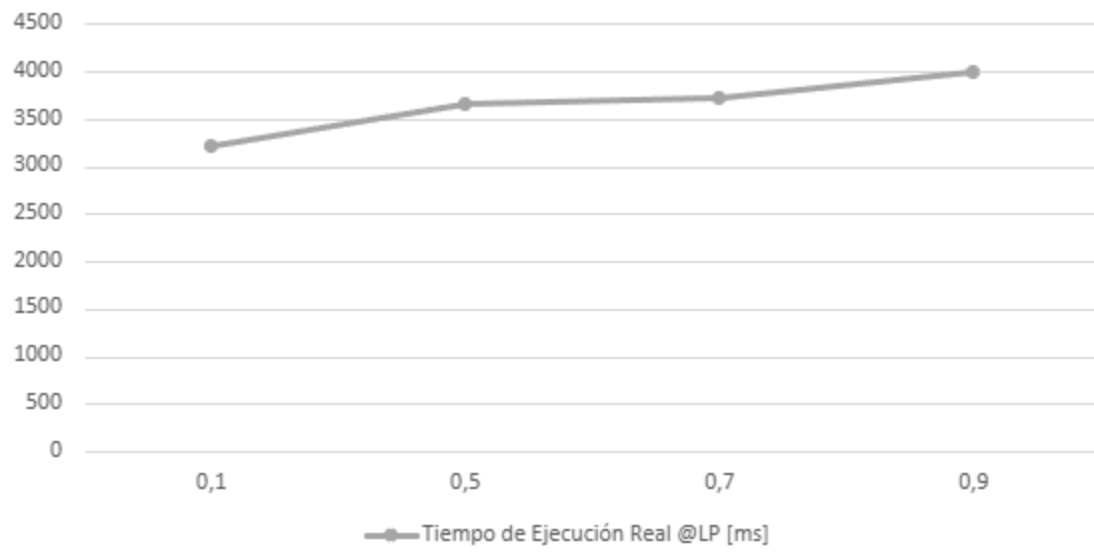
Consumo de Datos [kB]-Linear Probing

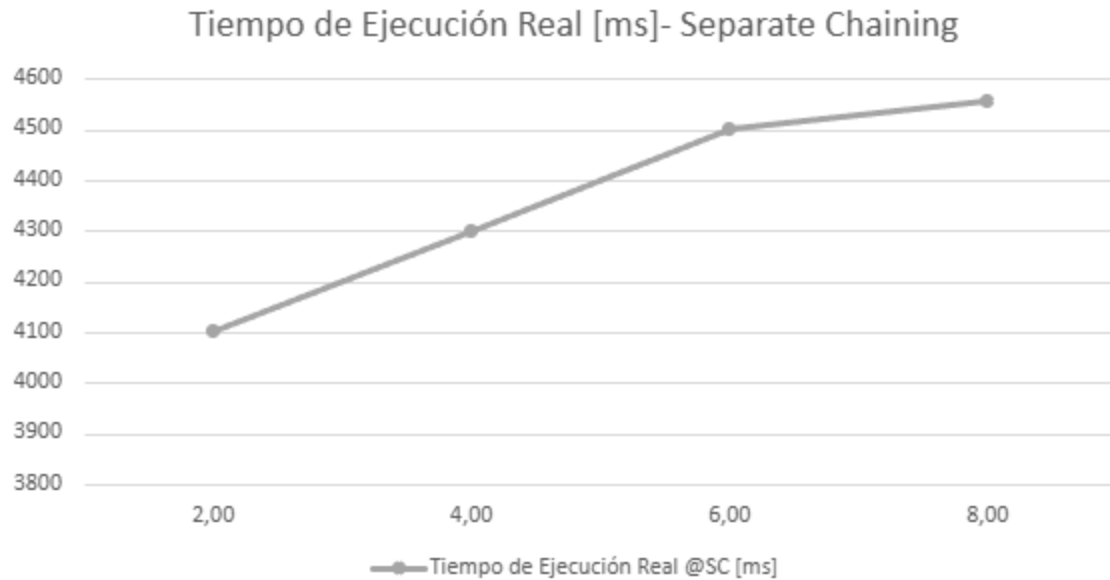


Consumo de Datos [kB]- Separate Chaining



Tiempo de Ejecución Real [ms]- Linear Probing





Preguntas de análisis

1. ¿Por qué en la función **getTime()** se utiliza **time.perf_counter()** en vez de otras funciones como **time.process_time()**?
time.perf_counter() mide el tiempo real de reloj que tú esperas sentado frente a la pantalla.
time.process_time() solo mide el tiempo que el procesador estuvo trabajando activamente, ignorando las pausas del sistema.
2. ¿Por qué son importantes las funciones **start()** y **stop()** de la librería **tracemalloc**?
Funcionan como un delimitador para aislar la medición de memoria. **start()** inicia el rastreo ignorando el consumo previo de Python, y **stop()** lo apaga al finalizar la función para evitar la sobrecarga del sistema y calcular la diferencia exacta de RAM consumida únicamente por el algoritmo evaluado.
3. ¿Por qué no se puede medir paralelamente el **uso de memoria** y el **tiempo de ejecución** de las operaciones?
Rastrear la memoria con **tracemalloc** interrumpe la ejecución en cada asignación para registrar datos, añadiendo un retraso enorme. Si mides el tiempo en simultáneo, obtendrás un tiempo alterado e inflado por la sobrecarga del rastreador.
4. Dado el número de elementos de los archivos (**large**), ¿Cuál sería el factor de carga para estos índices según su mecanismo de colisión?
En Separate Chaining, el factor de carga puede superar 1.0 (más del 100%) porque las sublistas anidadas albergan múltiples datos en una misma casilla. En Linear Probing nunca puede superar 1.0 (y suele limitarse a 0.75), pues cada celda solo guarda un dato y sin espacio libre la búsqueda entra en un ciclo infinito.

5. ¿Qué cambios percibe en el tiempo de ejecución al modificar el factor de carga máximo?
A mayor factor de carga, el tiempo individual de búsqueda aumenta debido al incremento de colisiones (recorrer más casillas consecutivas en Linear Probing o sublistas más largas en Separate Chaining). Sin embargo, un factor de carga muy bajo obliga a realizar rehash con mayor frecuencia, introduciendo picos de lentitud durante la carga masiva de datos.
6. ¿Qué cambios percibe en el consumo de memoria al modificar el factor de carga máximo?
Un factor de carga alto optimiza el uso de memoria RAM porque aprovecha mejor las casillas existentes antes de expandir el arreglo. Un factor de carga bajo incrementa el consumo total de memoria, ya que la tabla se duplica prematuramente dejando muchas celdas desaprovechadas.
7. ¿Qué cambios percibe en el tiempo de ejecución al modificar el esquema de colisiones? Si los percibe, describa las diferencias y argumente su respuesta.
Linear Probing es más rápido en factores de carga bajos gracias al aprovechamiento de la memoria caché al recorrer posiciones continuas. Separate Chaining ofrece tiempos de ejecución más estables y parejos a medida que la tabla se llena, aunque su velocidad base puede ser un poco menor debido al acceso disperso en memoria al recorrer las sublistas.
8. ¿Qué cambios percibe en el consumo de memoria al modificar el esquema de colisiones? Si los percibe, describa las diferencias y argumente su respuesta.
Linear Probing consume menos memoria estructural porque almacena las parejas llave-valor directamente en el arreglo principal. Separate Chaining requiere mayor memoria RAM debido a la sobrecarga de instanciar y mantener las sublistas o nodos adicionales en cada casilla de la tabla.